

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 12, Abgabe am 09.02.26 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Beachten Sie die geforderten Dateiformate und Dateinamen, die pro Aufgabe angegeben sind.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Hausaufgabe 1 (4 Punkte):

Achtung: Diese Aufgabe soll entweder als Textdatei `b12a1.txt` oder als PDF-Datei `b12a1.pdf` abgegeben werden. Bitte kennzeichnen Sie die Unteraufgaben.

Geben Sie für die folgenden Paare von Funktionen jeweils mit Begründung an, ob $f \in O(g)$ oder ob $g \in O(f)$ oder ob beides gilt. Begründen Sie Ihre Aussage jeweils wie folgt:

Um zu zeigen, dass $f \in O(g)$ geben Sie ein $c \in \mathbb{R}^+$ und ein $n_0 \in \mathbb{N}$ an, so dass $f(n) \leq c \cdot g(n)$ für alle $n > n_0$. Falls $f \notin O(g)$, müssen Sie keine Begründung angeben.

Das Kriterium gilt analog für $g \in O(f)$.

- (a) $f(n) = 6n^2$ und $g(n) = (n + 1)^2$
- (b) $f(n) = n^3 + 5$ und $g(n) = 12n^2$
- (c) $f(n) = 22n^3 + n^2$ und $g(n) = 3^n$

Hausaufgabe 2 (1 + 1 + 1 = 3

Hausaufgabe 3 (2 + 2 + 2 = 6 Punkte):

Achtung: Diese Aufgabe soll entweder als Textdatei `b12a3.txt` oder als PDF-Datei `b12a3.pdf` abgegeben werden. Bitte kennzeichnen Sie die Unteraufgaben.

Die folgenden Funktionen erwarten als Eingabe je entweder eine nicht-negative Ganzzahl n (d.h. $n \in \mathbb{N}$) oder eine Liste L . Geben Sie jeweils die Worst-Case-Laufzeit der Funktion, in Abhängigkeit von der Eingabe $n \in \mathbb{N}$ bzw. der Länge $n \in \mathbb{N}$ der Eingabe, an.

Die Worst-Case-Laufzeit beziehen sich auf die am schwächsten wachsende geeignete Funktion. Vereinfachen Sie diese so weit wie möglich, insbesondere müssen überflüssige Summanden und Faktoren vermieden werden.

Begründen Sie Ihre Angaben wie im folgenden Beispiel:

```
def programm0(n):  
    if n % 2 == 0:  
        return -1  
    j = 0  
    for i in range(n):  
        j += i  
    return j
```

Bei Eingabe von geraden Zahlen werden nur die ersten beiden Zeilen einmal ausgeführt. Bei Eingabe von ungeraden Zahlen wird die erste Zeile einmal ausgeführt und im Anschluss wird die for-Schleife n mal ausgeführt. Da

```

(a)    def programm1(n): # Eingabegroesse: n
        i = 1
        for j in range(n):
            i *= 2
        j = 1
        for k in range(i):
            j *= 2
        return j

(b)    def programm2(n): # Eingabegroesse: n
        i = 0
        while i <= n:
            i += 1
            if i ** 2 > 0:
                break
        return i

(c)    def programm3(L): # Eingabegroesse: n=len(L)
        n = len(L)
        y = []
        for i in range(n):
            for j in range(n):
                z = []
                k = 3
                while k < n:
                    pos = (i + j) % n
                    z += [type(pos)]
                    k += 1
                y.append(z)
        return y

```

Hausaufgabe 4 (2 + 2 + 3 = 7 Punkte):

Achtung: Geben Sie für diese Aufgabe zwei Dateien ab: Eine Textdatei **b12a4.txt** oder PDF-Datei **b12a4.pdf** für die Lösungen von Teilaufgaben (a) und (b) und die Begründung in (c). Außerdem eine Python-Datei **b12a4.py** für die Funktion aus (c).

Gegeben ist das folgende Programm, wobei `maxmult(L)` die Hauptfunktion ist:

```

def my_max(L):
    if len(L) == 0:
        return 0

    a = my_max(L[1:])
    return a if a > L[0] else L[0]

def maxmult(L):
    if len(L) < 2:
        return 0

    a = maxmult(L[1:])
    b = my_max(L[1:]) * L[0]
    return a if a > b else b

```

Die Funktion `maxmult(L)` erwartet eine Liste `L` von positiven Ganzzahlen. Zurückgegeben wird die größte Zahl, welche sich durch Multiplikation zweier Zahlen, die an verschiedenen Positionen in `L` stehen, ergeben kann, falls die Liste mindestens Länge 2 hat. Ansonsten wird 0 zurückgegeben.

Beispiel: Bei Eingabe `[2,6,4]` gibt `maxmult` die Zahl 24 zurück und bei Eingabe `[2,6,8,1,8]` gibt `maxmult` die Zahl 64 zurück.

- (a) Beschreiben Sie kurz, wie diese Zahl durch die Funktionen rekursiv berechnet wird.
- (b) Geben Sie die asymptotische Worst-Case Laufzeit von `maxmult(L)`, in Abhängigkeit von $n = \text{len}(L)$, mit Begründung an.
- (c) Schreiben Sie eine Funktion `maxmult_lin(L)`, welche die gleiche Funktion wie `maxmult(L)` berechnet, aber nur lineare Laufzeit benötigt. Begründen Sie, dass Ihr `maxmult_lin(L)` tatsächlich nur lineare Zeit benötigt.

H

Präsenzaufgaben

Präsenzaufgabe 1:

Zur Erinnerung: Für zwei Funktionen $f, g: \mathbb{N} \rightarrow \mathbb{N}$ schreiben wir $f \in O(g)$, wenn es $c \in \mathbb{R}^+$ und $n_0 \in \mathbb{N}$ gibt so dass für alle $n > n_0$ gilt, dass $f(n) \leq c \cdot g(n)$.

Geben Sie für die folgenden Paare von Funktionen jeweils mit Begründung an, ob $f \in O(g)$ oder ob $g \in O(f)$ oder ob beides gilt. Begründen Sie Ihre Aussage jeweils wie folgt:

Um zu zeigen, dass $f \in O(g)$ geben Sie ein $c \in \mathbb{R}^+$ und ein $n_0 \in \mathbb{N}$ an, so dass $f(n) \leq c \cdot g(n)$ für alle $n > n_0$. Falls $f \notin O(g)$, überlegen Sie sich warum solch ein Paar von c und n_0 nicht existieren kann.

Das Kriterium gilt analog für $g \in O(f)$.

- (a) $f(n) = 3n^3$ und $g(n) = n^3 + 24n^2$
- (b)

Präsenzaufgabe 3:

Geben Sie für die folgenden Funktionen f jeweils die (bis auf Umstellen der Operanden) eindeutige natürliche Funktion g an für die $f \in O(g)$ gilt, die am schwächsten anwächst und die so weit vereinfacht ist wie möglich. Hierbei bedeutet *natürlich*, dass alle Zahlen, die in g vorkommen, Ganzzahlen ≥ 0 sind.

Beispiel: Für $f(n) = 2n^3 + 1$, ist die korrekte Lösung $g(n) = n^3$.

(a) $f(n) = 4n \cdot 2n^2 + 34 \cdot 10^{24} + 22n^2$

(b) $f(n) = 3n^{12} \cdot 16n^{10} \cdot 7n^9 + 2^3 \cdot 3^n \cdot 4n^2$

Präsenzaufgabe 4:

Die folgenden Funktionen erwarten als Eingabe je entweder eine nicht-negative Ganzzahl n (d.h. $n \in \mathbb{N}$) oder eine Liste L . Geben Sie jeweils die Best-Case- und die Worst-Case-Laufzeit der Funktion, in Abhängigkeit von der Eingabe $n \in \mathbb{N}$ bzw. der Länge $n \in \mathbb{N}$ der Eingabe, an.

Der Best-Case für dieses Programm wird bei Eingabe von gerader Zahlen erreicht. In diesem Fall benötigt das Programm eine konstante Anzahl von Operationen, denn es werden nur die ersten beiden Zeilen je einmal ausgeführt. Der Best-Case ist also in $O(1)$. Der Worst-Case wird bei Eingabe von ungeraden Zahlen erreicht, die Laufzeit ist dann in $O(n)$. In diesem Fall werden die erste Zeilen einmal, und die Zeile in der Schleife n mal ausgeführt.

Für Funktionen mit Eingabe n müssen Sie in Ihrer Laufzeit-Analyse nur nicht-negative Ganzzahlen als Eingabe betrachten. Außerdem dürfen Sie in dieser Aufgabe davon ausgehen, dass eine einzelne Ausführung jeder Zeile nur eine konstante Anzahl von Operationen benötigt.

(a)

```
def programm1(n): # Eingabegroesse: n
    if n < 2:
        return False
    divisor = 2
    while divisor * 2 <= n:
        if n % divisor == 0:
            return False
        divisor += 1
    return True
```

(b)

```
def programm2(n): # Eingabegroesse: n
    i = 0
    if n <= 2:
        for j in range(2**n):
            i += 1
        return i
    else:
        for j in range(n):
            i += 1
        return programm2(n-1)
```

```
(c) def programm3(L): # Eingabegroesse: n=len(L)
    n = len(L)
    i = 0
    while n > 0:
        i += 1
        if n % 2 == 0:
            n = n // 2
        else:
            n = n // 4
    return i
```

Präsenzaufgabe 5:

Wir wollen im Folgenden Listen mit Integer-Einträgen nach Größe aufsteigend sortieren. Machen Sie sich dazu mit dem folgenden Sortieralgorithmus vertraut:

Insertion-Sort:

- Gehe iterativ die gesamte Liste durch und merke dir den Index des aktuellen Elements als Laufindex.
 - Das aktuelle Element wird mit den vorhergehenden Elementen verglichen, bis die passende Stelle gefunden ist. An dieser Stelle wird das Element dann eingefügt.
 - Die Menge vor dem aktuellen Laufindex wird als sortiert betrachtet, die Menge nach dem Index als unsortiert.
 - Solange noch als unsortiert geltende Elemente vorhanden sind, geht es mit dem nächsten Element weiter.
- (a) Schreiben Sie eine Funktion **insertion_sort**, welche eine Integer-Liste erwartet und diese mittels Insertion-Sort sortiert.
- (b) Analysieren Sie die asymptotische Best-Case- und Worst-Case-Laufzeit der Funktion, in Abhängigkeit von der Länge der Eingabeliste. Für welche unendliche Menge von Eingaben wird der Best-Case bzw. der Worst-Case erreicht?